



Angular Interview Questions - Theory & Conceptual Explanations



1. Change Detection Strategy

Explanation : Angular's change detection is a mechanism that checks for changes in data bindings and updates the DOM accordingly. It operates on two strategies:

Default Strategy: Triggers change detection for all components whenever any async event occurs (click, timer, HTTP response, etc.). This uses Zone.js to monkey-patch browser APIs and detect async operations. It checks from root component down to leaves (depth-first).

OnPush Strategy: Optimizes performance by only running change detection when:

1. Input reference changes (new object/array)
2. Component or its children emit events
3. Async pipe receives new value
4. Change detection is manually triggered

Interview Points to Emphasize:

- OnPush can improve performance by 30-70% in large applications
- Always use immutable data with OnPush (return new objects/arrays)
- Async pipe automatically triggers change detection
- markForCheck() vs detectChanges(): First schedules check for next cycle, second runs immediate check
- Zone-less applications require manual change detection



2. Reactive Programming

Explanation : Reactive programming in Angular uses RxJS (Reactive Extensions for JavaScript) to handle asynchronous operations through observable streams. Key concepts:

Observables: Lazy collections that emit values over time. Unlike promises, they can emit multiple values and can be cancelled.

Subjects: Special observables that act as both observable and observer. **Types:**

- **Subject:** No initial value, multicast
- **BehaviorSubject:** Has initial value, multicasts current value to new subscribers
- **ReplaySubject:** Replays specified number of previous values
- **AsyncSubject:** Emits only last value on completion

Operators: Pure functions for transforming streams:

- **Creation:** of, from, interval
- **Transformation:** map, switchMap, mergeMap, concatMap
- **Filtering:** filter, debounceTime, distinctUntilChanged
- **Combination:** combineLatest, withLatestFrom, forkJoin

Interview Points to Emphasize:

- switchMap cancels previous inner observable (good for search)
- mergeMap runs inner observables in parallel
- concatMap runs inner observables sequentially
- Always unsubscribe to prevent memory leaks (use async pipe, takeUntil pattern)
- Hot vs Cold observables



3. Dependency Injection Hierarchy

Explanation : Angular's DI system is hierarchical and follows the component tree structure:

Hierarchy Levels:

1. **Root/Platform Injector:** Services provided in root or platformBrowser
2. **Module Injector:** Services provided in NgModule providers
3. **Component Injector:** Services provided in component providers

Resolution Order: When Angular needs a dependency:

1. Checks component's providers
2. Moves up to parent component
3. Continues to root component
4. Checks module providers
5. Finally checks root injector

Interview Points:

- providedIn: 'root' makes service singleton application-wide
- Component providers create new instances per component tree
- Use @Optional() decorator for optional dependencies
- Use @Self() to only look in current injector
- Use @SkipSelf() to skip current injector



4. Standalone Components

Explanation : Introduced in Angular 14+, standalone components eliminate the need for NgModules, allowing:

- Self-contained components with their own dependencies
- Lazy loading at component level
- Simpler project structure
- Gradual migration from modules

Key Features:

- standalone: true in component decorator
- Direct imports of dependencies
- Can bootstrap standalone components
- Can be mixed with module-based components

Interview Points:

- Migrate strategy: Start with leaf components
- Still need modules for some libraries
- Better tree-shaking potential
- Future direction of Angular



5. State Management

Explanation : State management patterns in Angular:

1. Services with Subjects (Simple Apps):

- Use BehaviorSubject in services
- Component-level state management
- Good for small to medium apps

2. NgRx/Redux Pattern (Complex Apps):

- Single source of truth
- State is read-only
- Changes via pure functions (reducers)
- Predictable state changes
- DevTools for debugging

3. Component Store (NgRx/Component):

- Scoped to feature/section
- Lighter than full NgRx
- Good for medium complexity

4. Signals (Angular 16+):

- Fine-grained reactivity
- No need for observables in simple cases
- Better performance with zone-less

Interview Points:

- Choose pattern based on app complexity
- NgRx has steep learning curve but great for large teams
- Don't over-engineer small apps
- Consider Akita, NGXS as alternatives



6. Zone-less Angular & Performance

Explanation : Zone-less Angular removes Zone.js dependency, requiring manual change detection:

Benefits:

- Smaller bundle size (~30KB less)
- No unnecessary change detection cycles
- Better control over when updates happen
- Improved performance in high-frequency events

Drawbacks:

- Manual change detection management
- More complex code
- Need to handle all async operations manually

Performance Optimization Techniques:

1. OnPush Change Detection
2. TrackBy in ngFor
3. Pure Pipes
4. Lazy Loading
5. Virtual Scrolling
6. Web Workers for CPU-intensive tasks
7. AOT Compilation

Interview Points:

- Zone.js patches async APIs (setTimeout, addEventListener, etc.)
- runOutsideAngular() to prevent change detection
- Use ChangeDetectorRef.detach() to detach component
- Profile with Angular DevTools



7. Lifecycle Hooks Order

Explanation : Understanding lifecycle hook order is crucial:

Creation Phase:

1. `constructor()` - Dependencies injected
2. `ngOnChanges()` - Input bindings initialized (if any)
3. `ngOnInit()` - Component initialized
4. `ngDoCheck()` - Custom change detection
5. `ngAfterContentInit()` - Content projected
6. `ngAfterContentChecked()` - Content checked
7. `ngAfterViewInit()` - View children initialized
8. `ngAfterViewChecked()` - View checked

Update Phase (Repeats):

- `ngOnChanges()` (if inputs change)
- `ngDoCheck()`
- `ngAfterContentChecked()`
- `ngAfterViewChecked()`

Destruction:

- `ngOnDestroy()` - Cleanup

Interview Points:

- `ngOnInit` vs `constructor`: Use `constructor` for DI, `ngOnInit` for initialization logic
- `ngAfterViewInit` for DOM access
- `ngOnDestroy` for cleanup (unsubscribe, remove event listeners)
- Avoid heavy operations in hooks that run frequently



8. Reactive Forms vs Template-Driven

Explanation :

Reactive Forms:

- Model-driven approach
- Form created in component class
- Synchronous validation
- Immutable data flow
- Better for complex forms
- Easier unit testing

Template-Driven Forms:

- Template-driven approach
- Form created in template
- Asynchronous validation
- Two-way binding with ngModel
- Simple for basic forms
- Less code

Interview Points:

- Reactive forms are preferred for complex scenarios
- Use FormBuilder for cleaner syntax
- Custom validators are easier in reactive forms
- Async validators for server-side validation



8. Reactive Forms vs Template-Driven

Explanation :

Interceptors are middleware that process HTTP requests and responses:

Common Use Cases:

1. Authentication: Add auth headers
2. Logging: Log requests/responses
3. Caching: Cache responses
4. Error Handling: Global error handling
5. Loading Indicators: Show/hide loading
6. Request Transformation: Modify requests

Execution Order:

Request: Interceptor1 => Interceptor2 => Server

Response: Server => Interceptor2 => Interceptor1 => Subscriber

Interview Points:

- Interceptors are services implementing `HttpInterceptor`
- Use `HttpInterceptorFn` for functional interceptors (Angular 15+)
- Can modify request/response or create new ones
- Handle errors in `catchError` operator
- Use `HttpContext` to pass metadata



8. Reactive Forms vs Template-Driven

Explanation :

Lazy loading loads feature modules on-demand:

Benefits:

- Faster initial load time
- Better performance
- Code splitting
- Reduced bundle size

Implementation:

1. Module-based: `loadChildren: () => import()`
2. Component-based: `loadComponent: () => import()` (Angular 14+)
3. Standalone: Direct component lazy loading

Preloading Strategies:

- NoPreloading: Only loads on navigation
- PreloadAllModules: Loads all after initial
- Custom Preloading: Selective preloading

Interview Points:

- Use route guards for authentication
- Preload frequently used modules
- Consider route-based chunk splitting
- Monitor bundle sizes